

CE2007

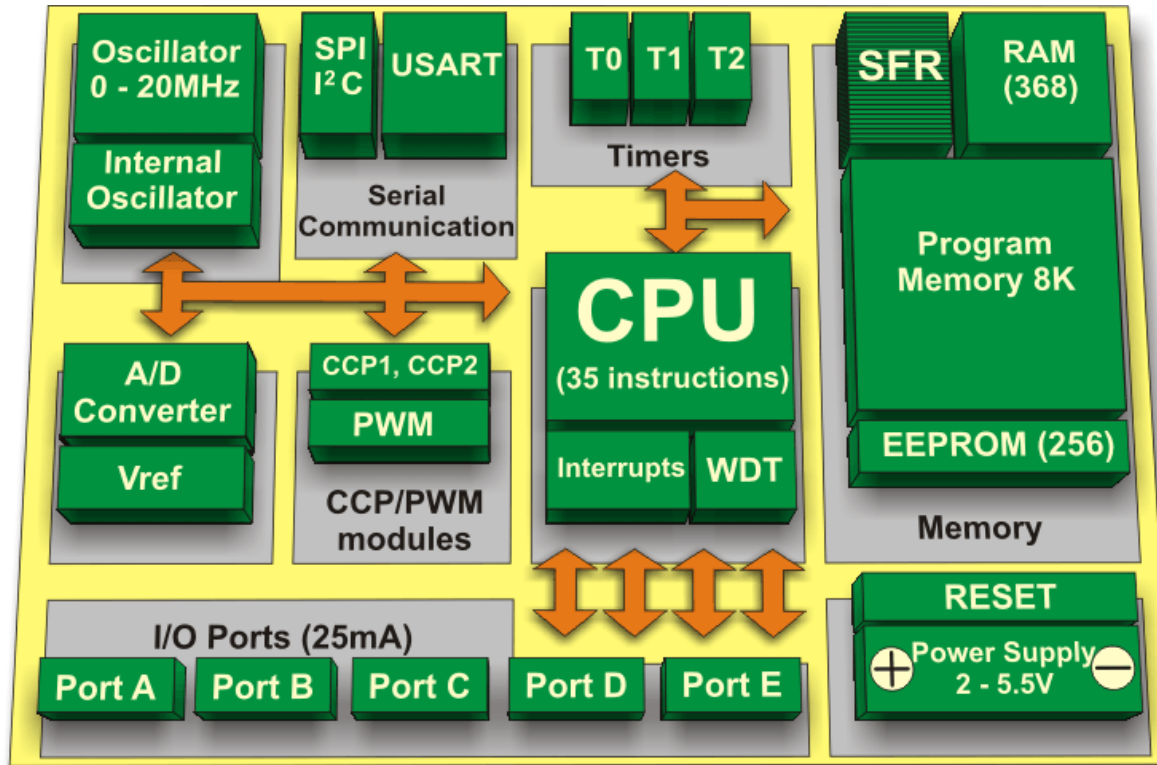
Microprocessor-based System Design

Peripherals and Interfaces

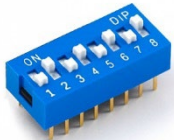
Part 1

I/O Devices for Embedded Systems

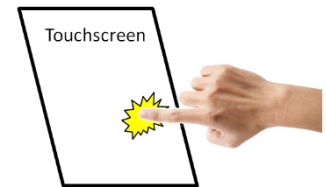
Microcontroller



Pushbutton
switch



Switches



LCD



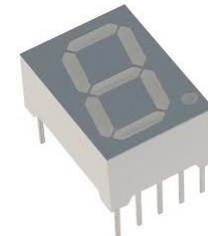
Keypad



LEDs



Dot Matrix LED



7 segment display

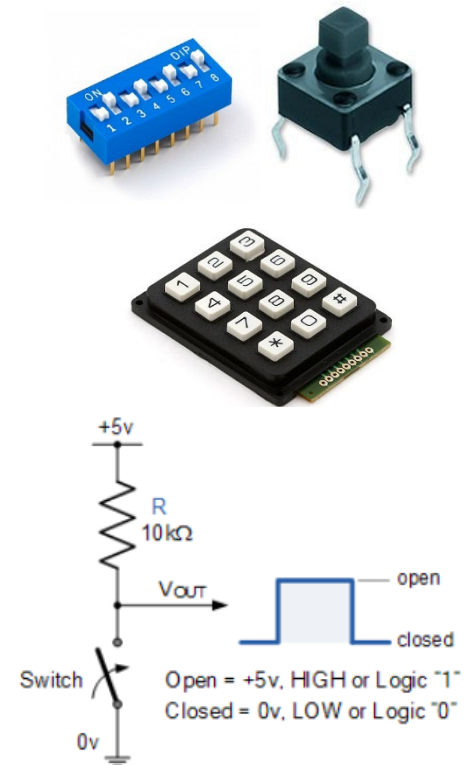
Topics Covered

- Switches
- Keypads
 - Keypad Scan Technique
- Debounce
 - Hardware Debounce Circuits
 - Software Debounce Techniques
- Timers and Counters

Switches and Keypads

Switch

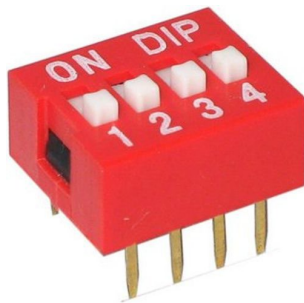
- Mechanical ON-OFF toggle switches, push-button switches or key switches are popular input devices because of their **low cost** and ease of **interfacing**.
- Most common way of interfacing a switch to an electronic circuit is via a pull-up resistor to the supply voltage.
 - When the switch is open, a logic “1” is given as the output signal.
 - When the switch is closed the output is grounded at 0v, or a logic “0” is given as the output.



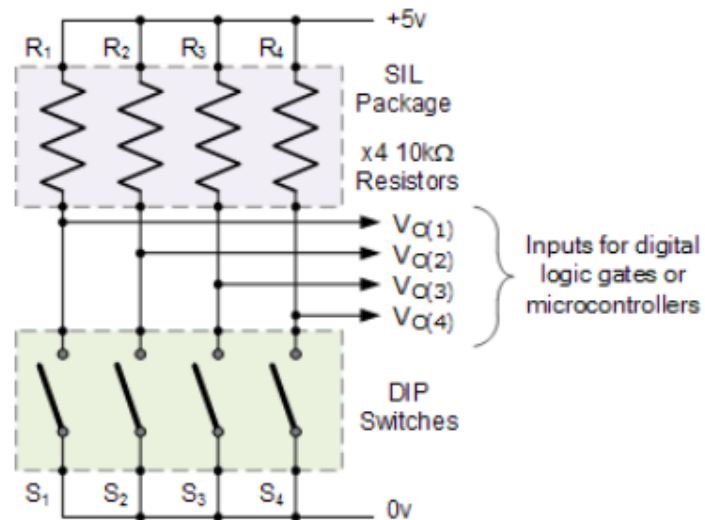
Source: <http://www.electronics-tutorials.ws/io/input-interfacing-circuits.html>

DIP Switch

- DIP or Dual-in-line Package switches are individual switches that are grouped together as four or eight switches within a single package.
- Each switch within a DIP switch package normally indicates one of two conditions by its ON-OFF status and a four switch DIP package will have four outputs.



Source:
<http://www.emartee.com/product/41712/DIP-Switch-4-Position>



Source: <http://www.electronics-tutorials.ws/io/input-interfacing-circuits.html>

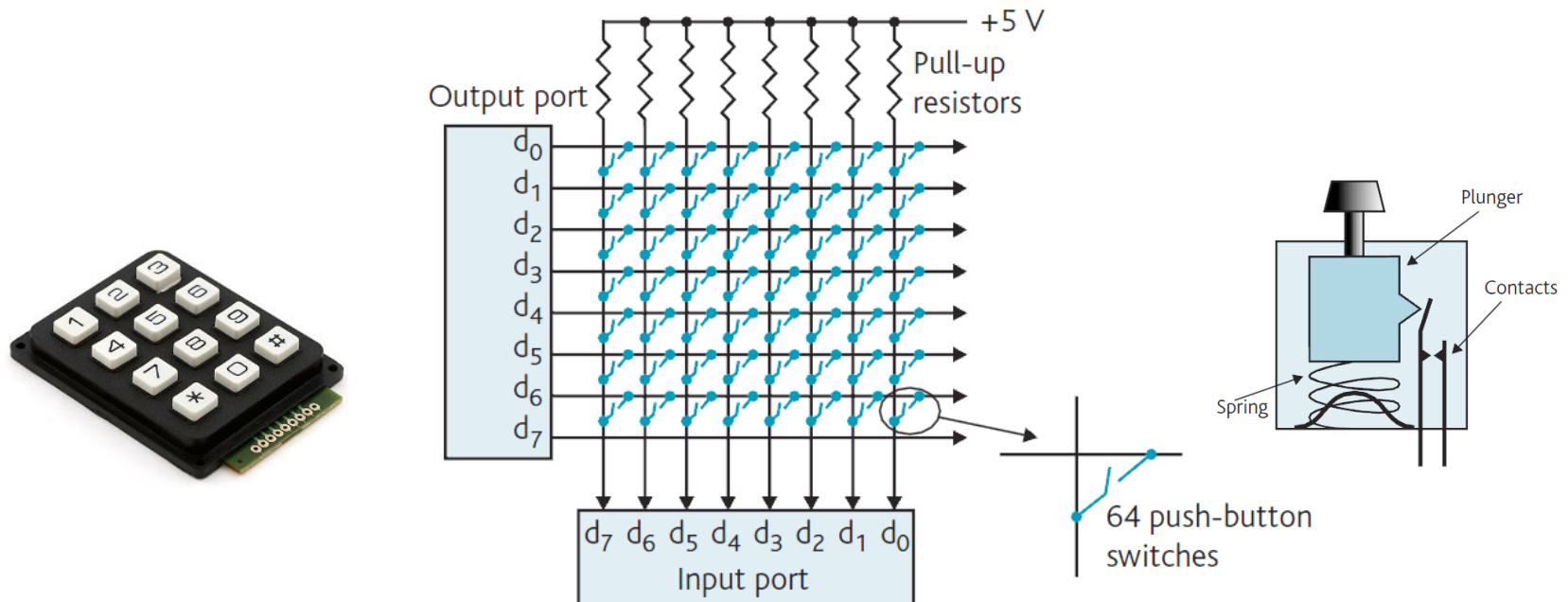
-
- 8-bit input port
- Toggle switches are data source.
- VCC
- 10KΩ
- 74ALS244
- 1A₁ 1Y₁
1A₂ 1Y₂
1A₃ 1Y₃
1A₄ 1Y₄
2A₁ 2Y₁
2A₂ 2Y₂
2A₃ 2Y₃
2A₄ 2Y₄
- Data Bus
- SEL
- I/O port address decoded to SEL
- When tri-states are enabled, microprocessor can read state of toggle switches into AL (using IN instruction).

<http://www.datasheetdir.com/DM74ALS244ASJX+Buffers-Drivers>

Peripherals and Interfaces – Part 1

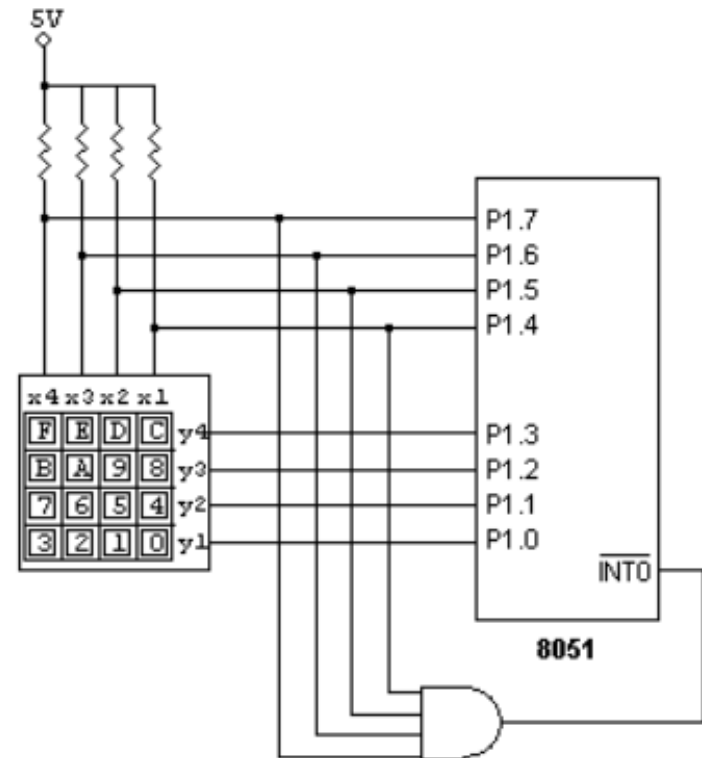
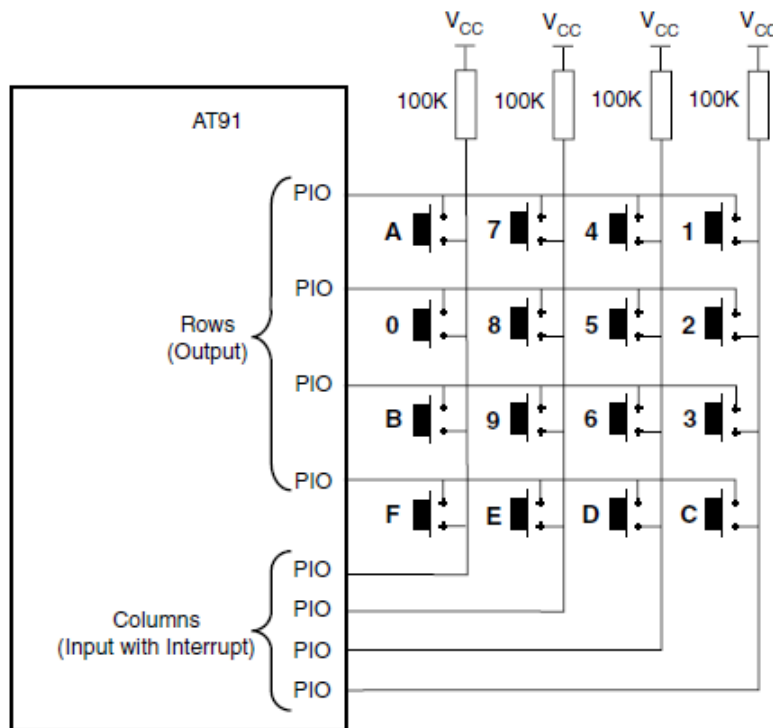
Keypad

- A keypad consists of a matrix of push-button switches.
- Come in a variety of sizes and configurations. The most basic would be the 4x4 matrix keyboard.
- In the default state (all switches open) there is no connection between the rows and columns. When a switch is pressed, a connection between the switch's row and the switch's column is made.



Interfacing Keypad to Microcontroller

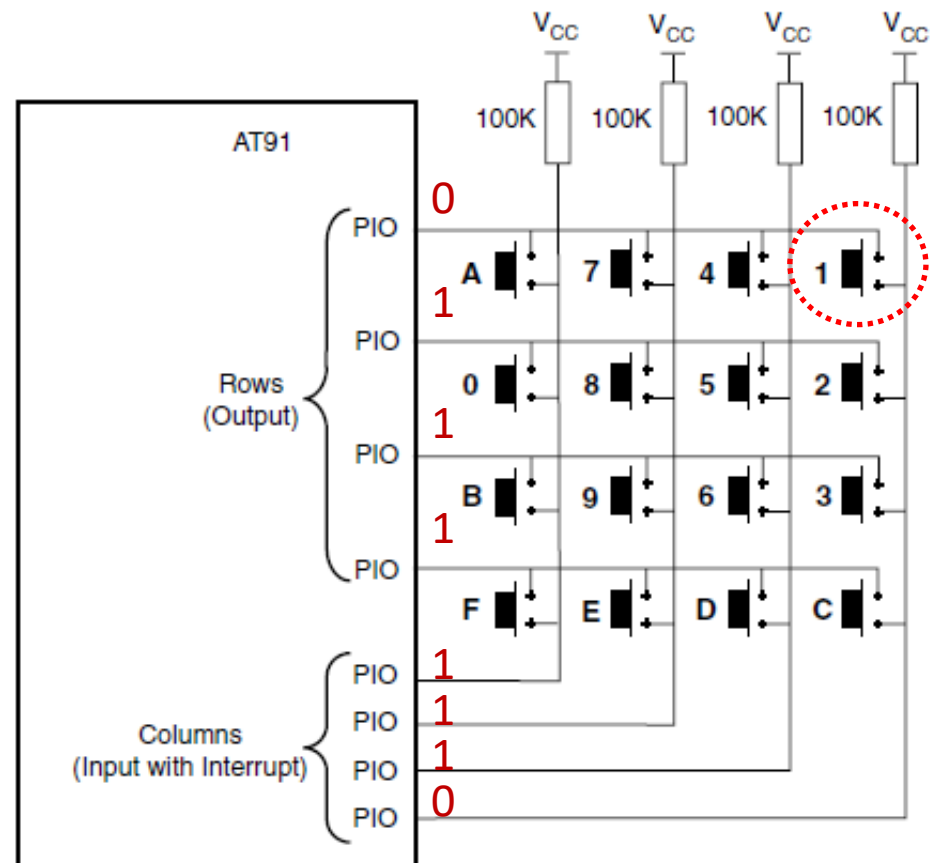
- The easiest and straight-forward means of connection would be through direct I/O lines.
- Four lines for the Rows to be initially configured as outputs. Four lines for the Columns to be configured as inputs (Interrupts Optional).



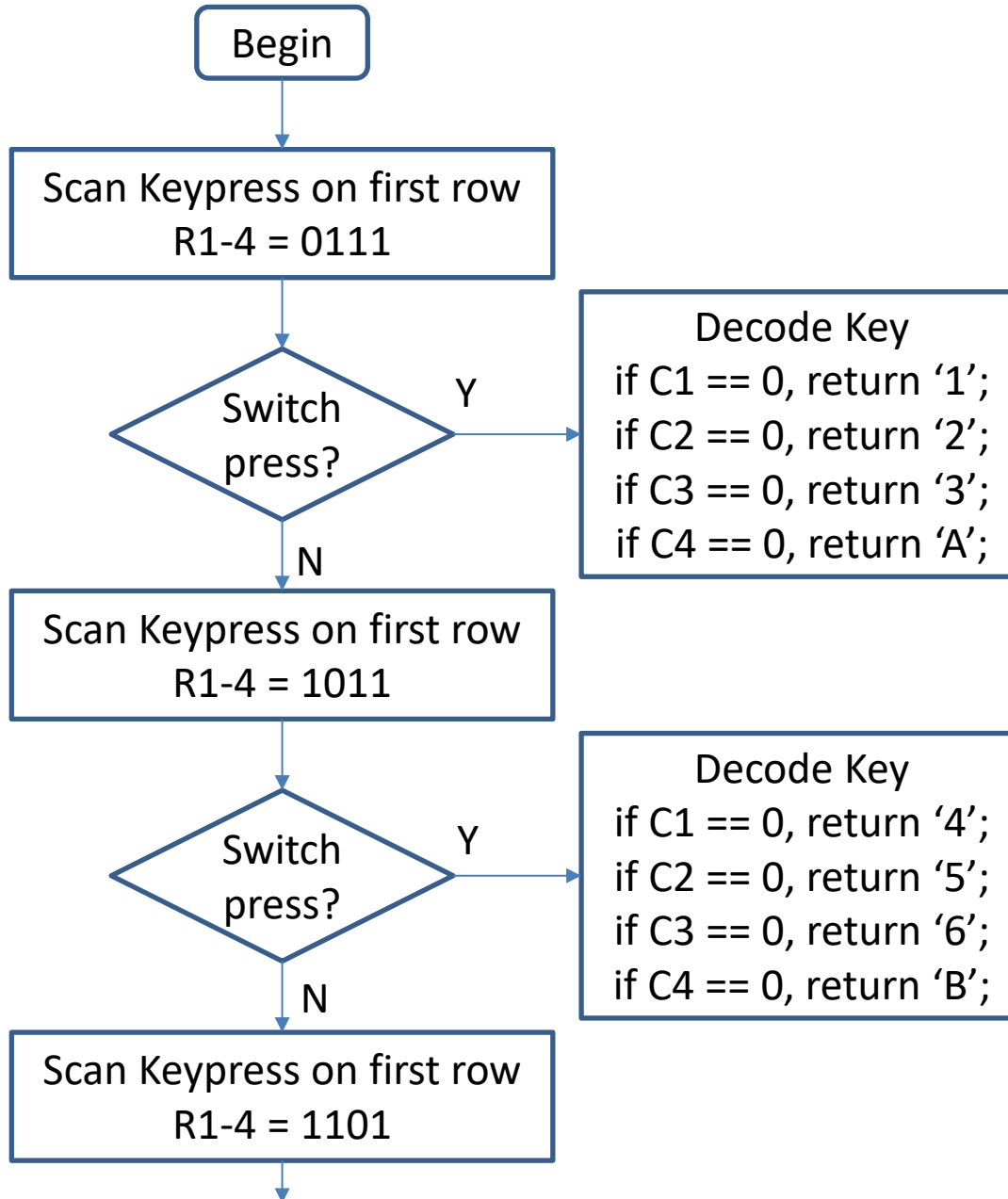
Source:
<http://www.edsim51.com/8051Notes/interfacing.html>

Keypad Scan Technique

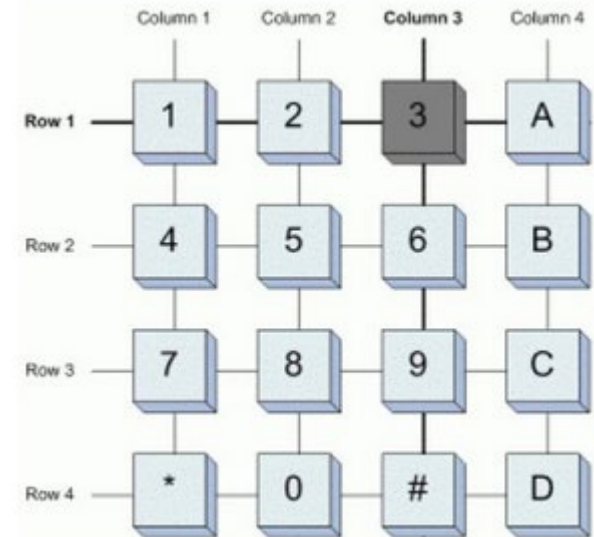
- The four vertical input lines are each terminated in a resistor connected to VCC, so that these lines are pulled HIGH. When no key is pressed, input port will read 1111.
- Suppose now the output port puts the binary value 01111 onto its four output lines.
- If top right-hand key pressed, the CPU reads from its input port 1110.



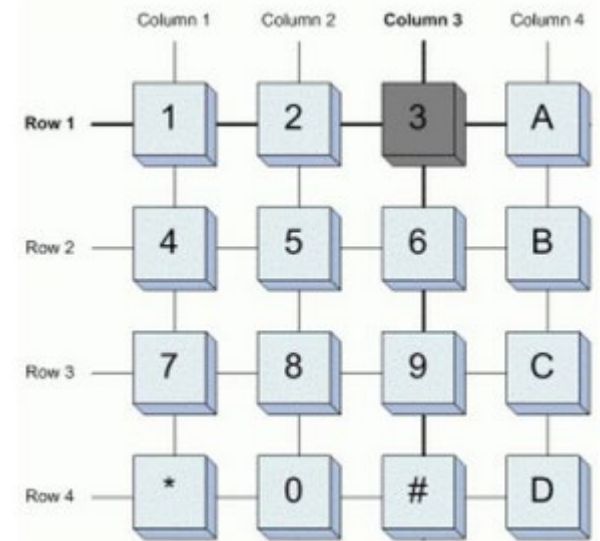
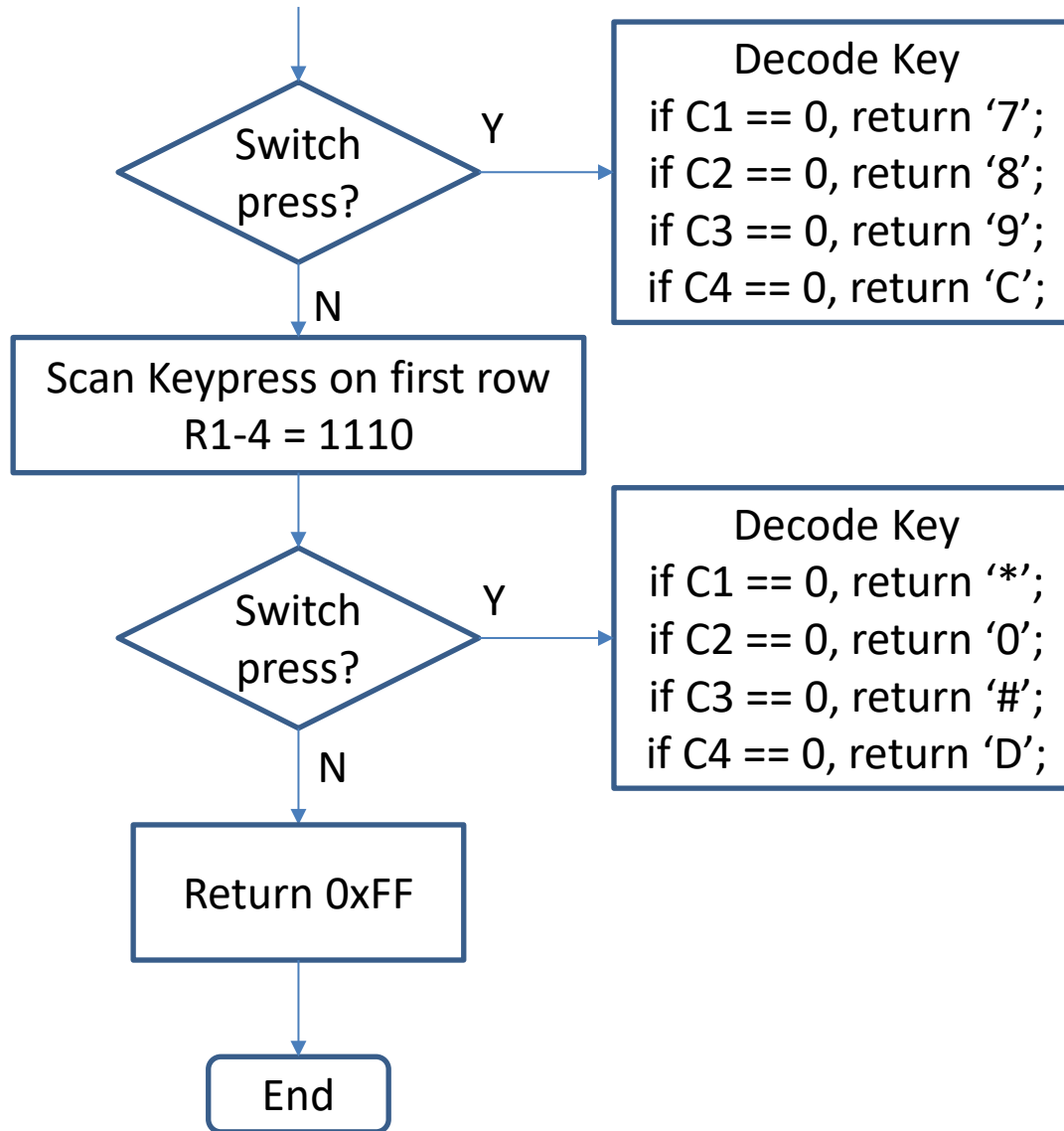
Keypad Scan and Decode Algorithm



```
keypad_wait (void) {  
    unsigned char key = 0xFF;  
  
    do {  
        key = keypad_scan_dec();  
    } while (key == 0xFF);  
  
    return key;  
}
```



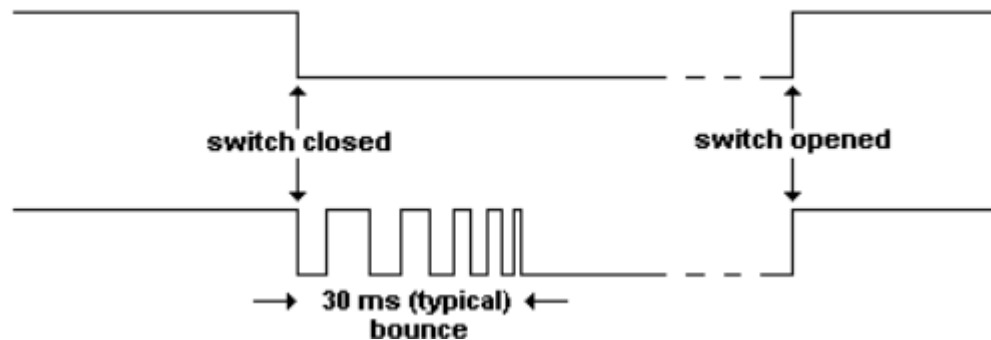
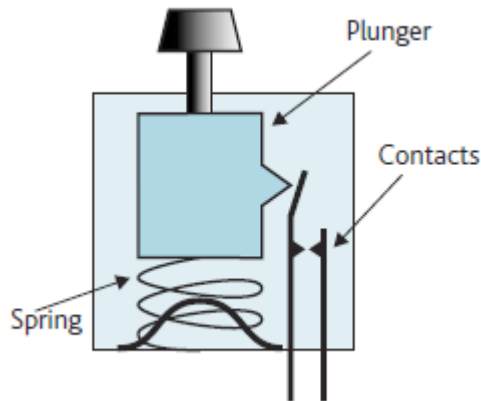
Keypad Scan and Decode Algorithm (Cont.)



Debounce

Switch Bounce

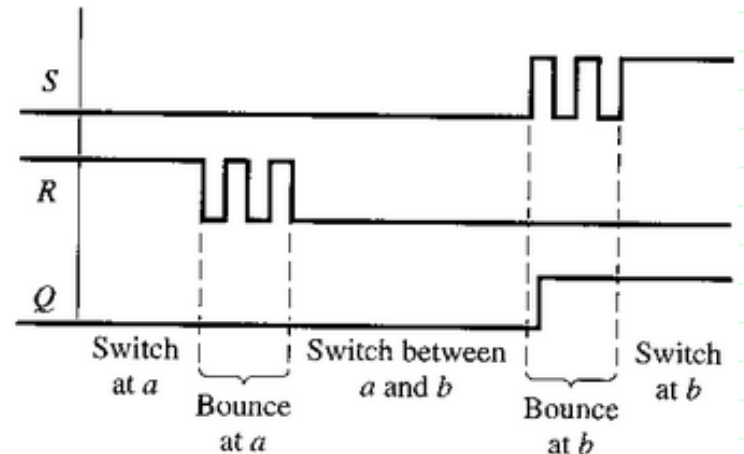
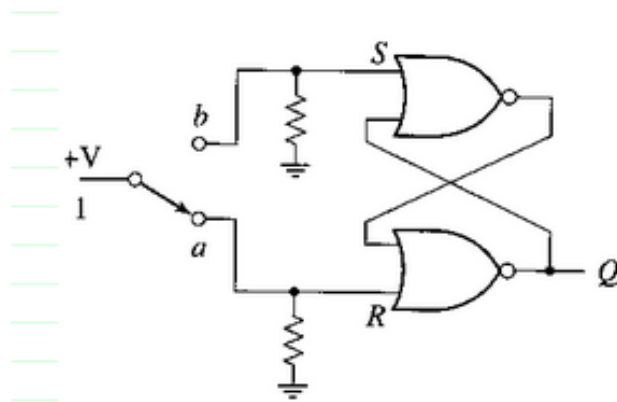
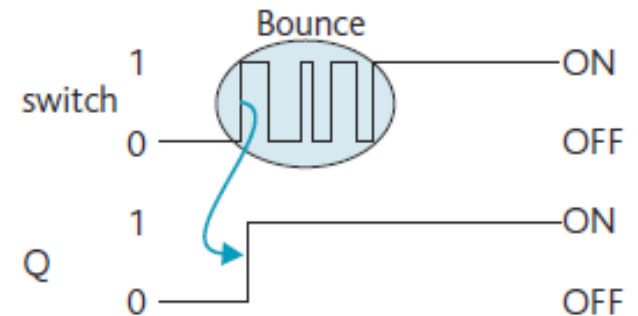
- Switches or Keypads suffer from **Switch Bounce**, which produces a series of pulses when contact is made.
- This can be registered as a series of ON and OFF signals lasting several milliseconds instead of just the one intended single and positive switching action.
- This problem can be solved using **Hardware Debounce Circuits (SR and RC Debounce)** or **Software Debounce**



Source: <http://www.edsim51.com/8051Notes/interfacing.html>

SR Debounce Circuit

- The effects of switch bounce can be eliminated by connecting the switch to the S input of an SR Flip-Flop.
- When the switch first closes, the Flip-Flop is set and its Q output goes high. Even if the contacts bounce and S goes low, the Q output remains high.

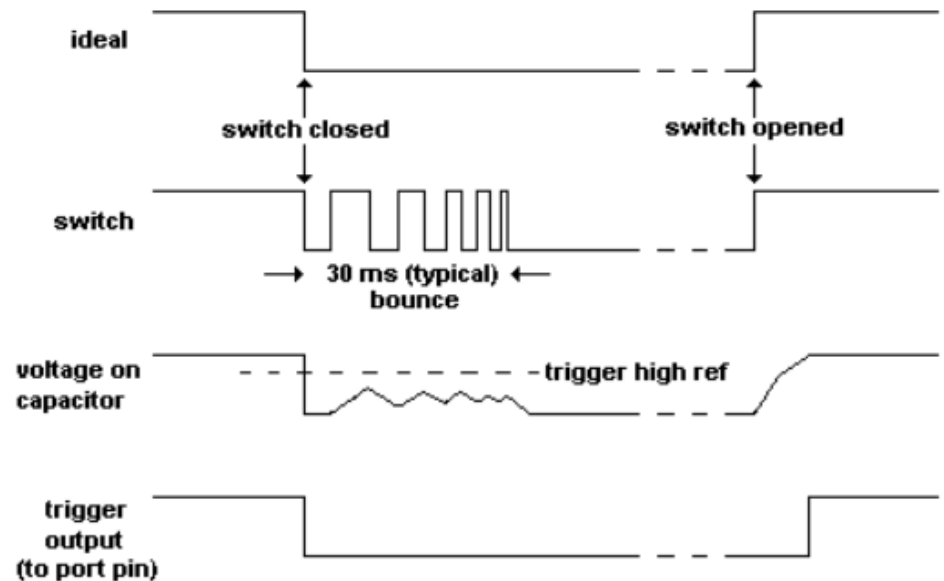
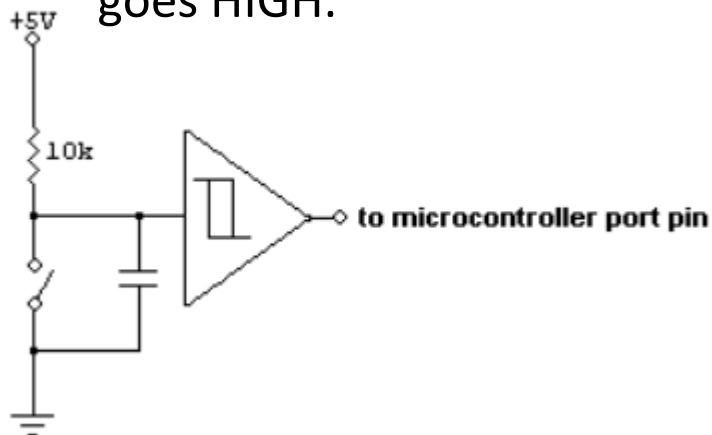


Source:

<http://yschen.ee.yzu.edu.tw/Courses/972/PostView1.asp?semester=972&coursenm=EE112B&codename=33>

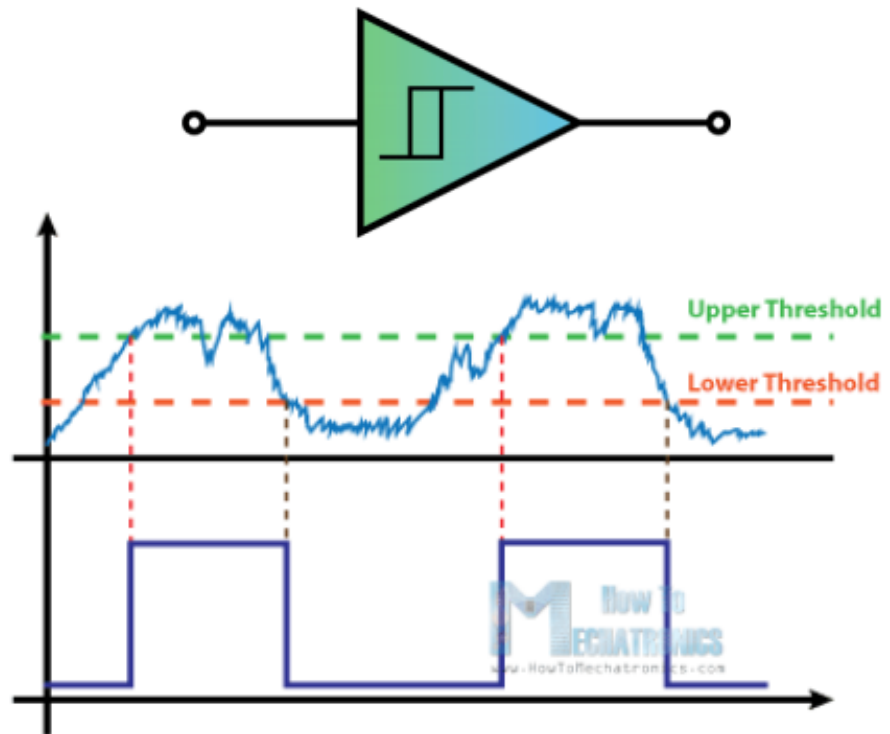
RC Debounce Circuit

- When the switch is closed, the capacitor voltage drops to 0 V.
- When the switch bounces open, the capacitor begins to charge back up to 5V, but due to the RC constant, it cannot charge beyond the Schmitt trigger's HIGH reference voltage before the switch bounces closed again.
- Once the switch closes, the capacitor again drops to 0V. Since the voltage on the capacitor never goes beyond the trigger's HIGH reference voltage during the switch bounce, the output of the trigger remains LOW.
- When the switch is physically opened, the capacitor has plenty of time to charge beyond the trigger's HIGH reference voltage, at which point the trigger output goes HIGH.



Schmitt Trigger

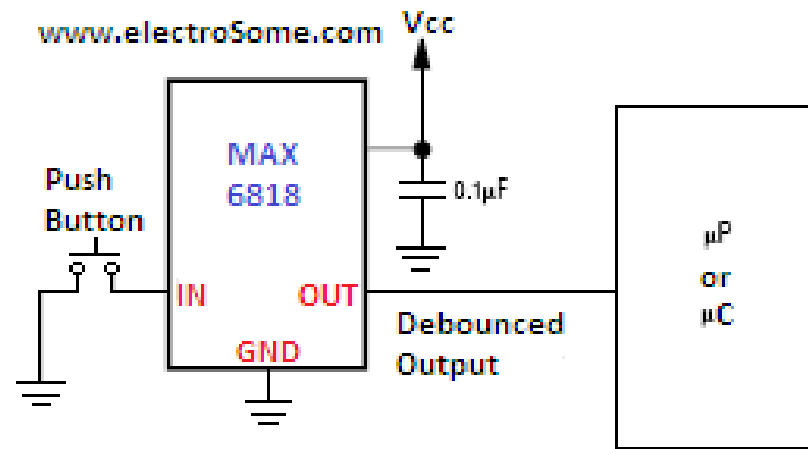
- The Schmitt Trigger is a logic that provides hysteresis or two different threshold voltage levels for rising and falling edge.
- This is useful to get square wave signals in the presence of noisy input signals.
- When the input is higher than a chosen threshold, the output is high.
- When the input is below a different (lower) chosen threshold the output is low.
- When the input is between the two levels the output retains its value.



Source: <http://howtomechatronics.com/how-it-works/electrical-engineering/schmitt-trigger/>

Switch Debounce IC

- Switch debouncing can also be done by specialized IC.
- Few and rarely used:
 - MAX6816, MAX 6817, MAX 6818
 - MC 14490



Switch Debouncing using IC MAX6816

Software Debounce Techniques

- Implemented in software and runs on the processor.
 - Low cost and flexible.
 - Important to minimize CPU overhead.

- Basic principle and methods:
 - Wait and See
 - Sample the switch signals and filter out glitches if any.
 - Counting
 - Shift Register

Wait and See

- Easiest approach is to wait for a delay time whenever button press is detected.
- After delay time, we can proceed to check which button has been pressed.

```
Set Return_Value = !Switch_Pressed
```

```
If (Switch_Pin == 0) // Switch is pressed
```

```
{
```

```
    WAIT (DEBOUNCE_PERIOD);
```

```
    If (Switch_Pin == 0) // If switch is still pressed
```

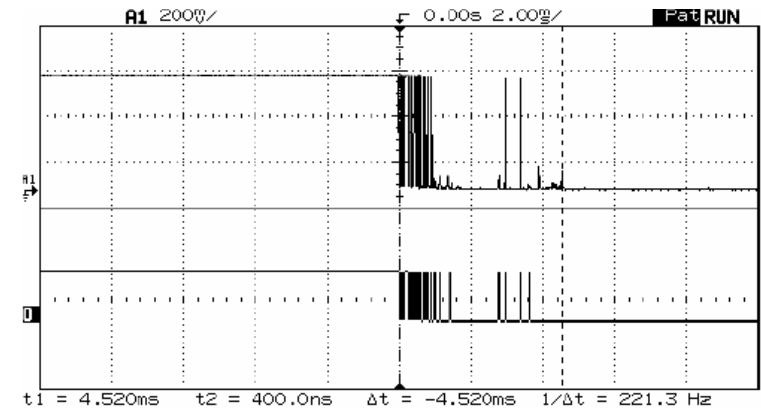
```
{
```

```
        Return_Value = Switch_Pressed;
```

```
}
```

```
}
```

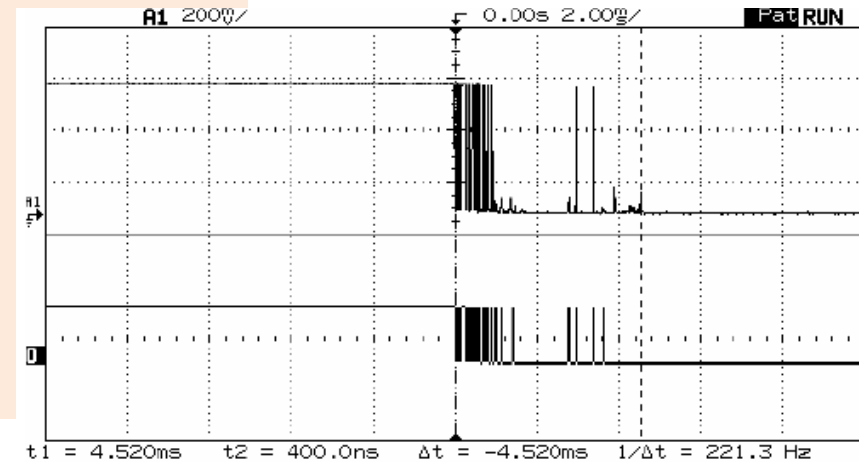
```
return (Return_Value);
```



Counting

- Look for N consecutive stable readings of the switch.

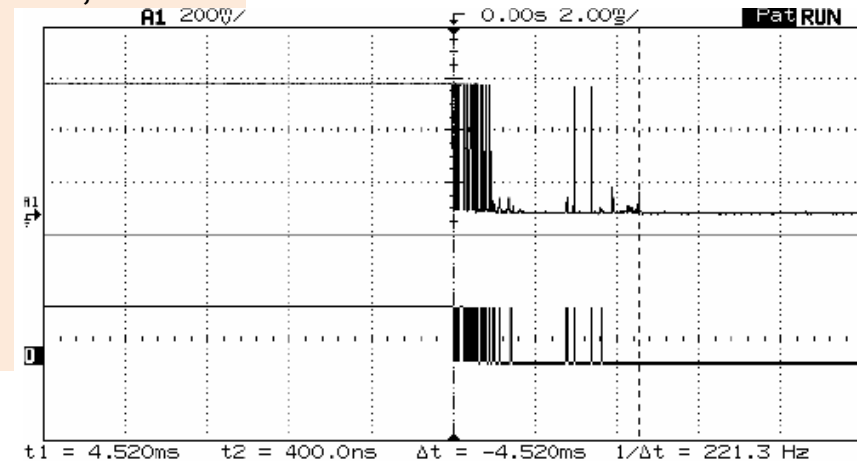
```
1. Set Count = 0
2. Using a timer, set up a sampling period (say 1 ms)
3. At each sampling event {
4.   If (Switch_Pin == 1) {
5.     Count = 0;
6.     Return_Value = !Switch_Pressed;
7.   }
8.   else
9.     Count++;
10.  end if
11.  If (Count ==  $N$ )
12.    Return_Value = Switch_Pressed;
13.  end if
14. }
```



Shift Register

- Similar to Counting method, but uses a shift register. More suitable for assembly programming.
- The following algorithm assumes unsigned 8-bit register is used.

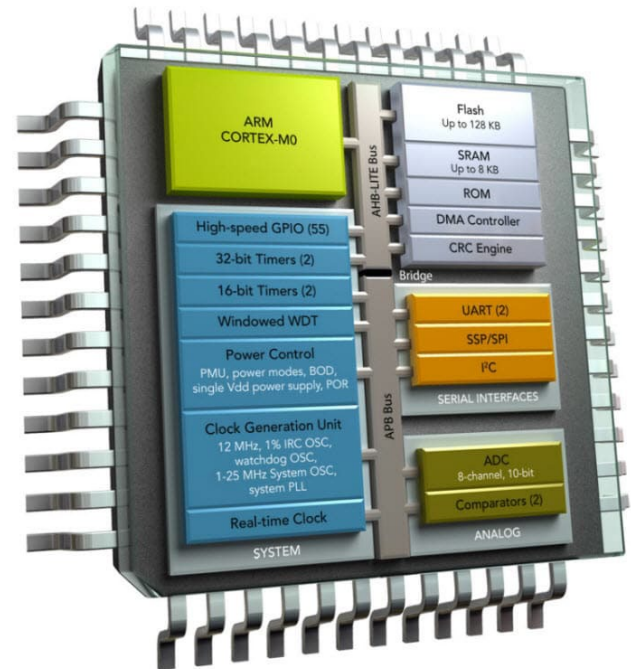
```
1. Set shift register variable to 0xFF
2. Using a timer, set up a sampling period (say 1 ms)
3. At each sampling event {
4.     Left shift variable;
5.     Set LSB of variable to current switch value;
6.     If (shift register == 0)
7.         Return_Value = Switch_Pressed;
8.     else
9.         Return_Value = !Switch_Pressed;
10.    end if
11. }
```



Timers and Counters

Timers and Counters

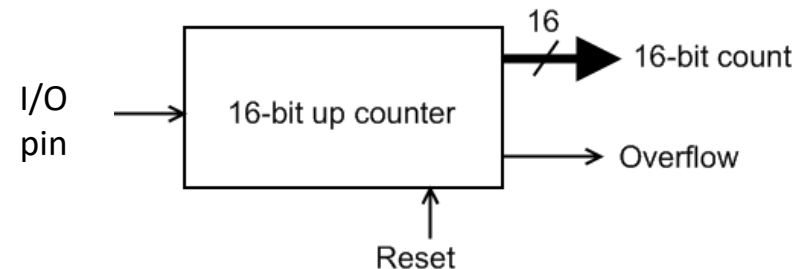
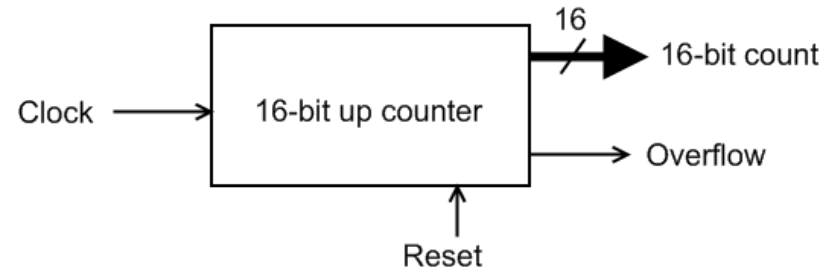
- Most embedded applications need to perform a variety of precision timer functions including:
 - Generate events at specific times
 - Determining the duration between two events
 - Count events.
- This can be achieved with the help of hardware timers/counters.
- The hardware timer/counter runs independently. It can count inputs or clock pulses concurrently while the microcontroller is executing the main program.



Source: <http://www.mouser.com/new/nxp-semiconductors/nxplpc1200/>

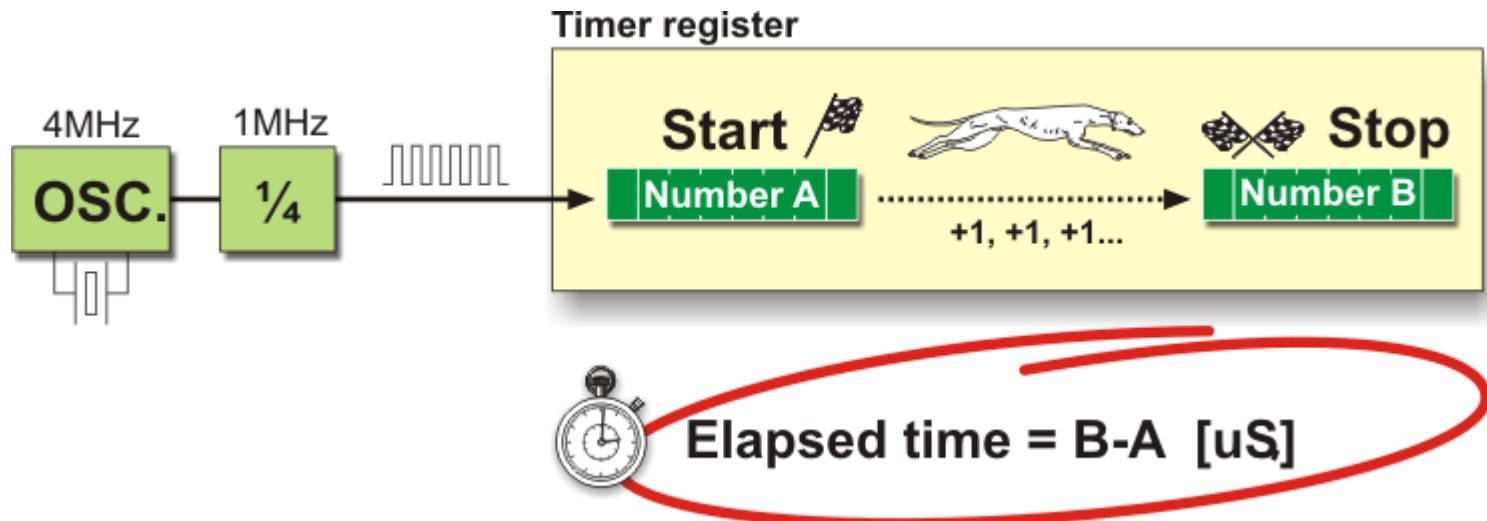
Timers vs. Counters

- Main component of a hardware timer/counter is a free running binary counter that increments at each incoming timing pulse.
- Timer:
 - Incoming pulse is a clock source with fixed frequency from the crystal oscillator.
 - Allows us to make accurate measurement of time interval by counting the pulses from the clock.
- Counter:
 - Incoming pulse is provided through external I/O pin of microcontroller.
 - Allows us to count external pulses which may appear at random intervals.
 - These pulses could be from a push button, sensors, etc.



How Does the Timer Work?

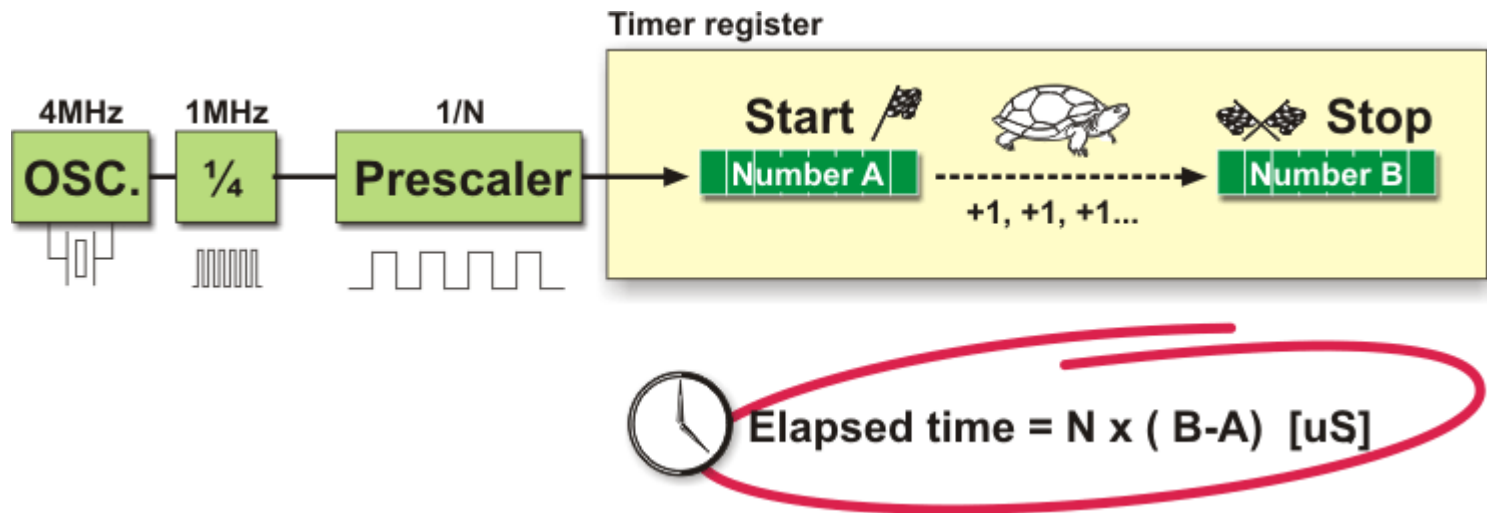
- Pulses are used to increment the number stored in the timer register.
- If the Timer register consists of 8 bits, the largest number it can store is 255. Based on the current oscillator setting below we can get a maximum timing interval of 256us.
- To measure longer periods of time, we can use:
 - Pre-scaler
 - Interrupts



Source: <https://learn.mikroe.com/ebooks/picmicrocontrollersprogramminginassembly/front-matter/introduction-to-the-world-of-microcontrollers/>

Using Pre-scaler to Measure Longer Periods

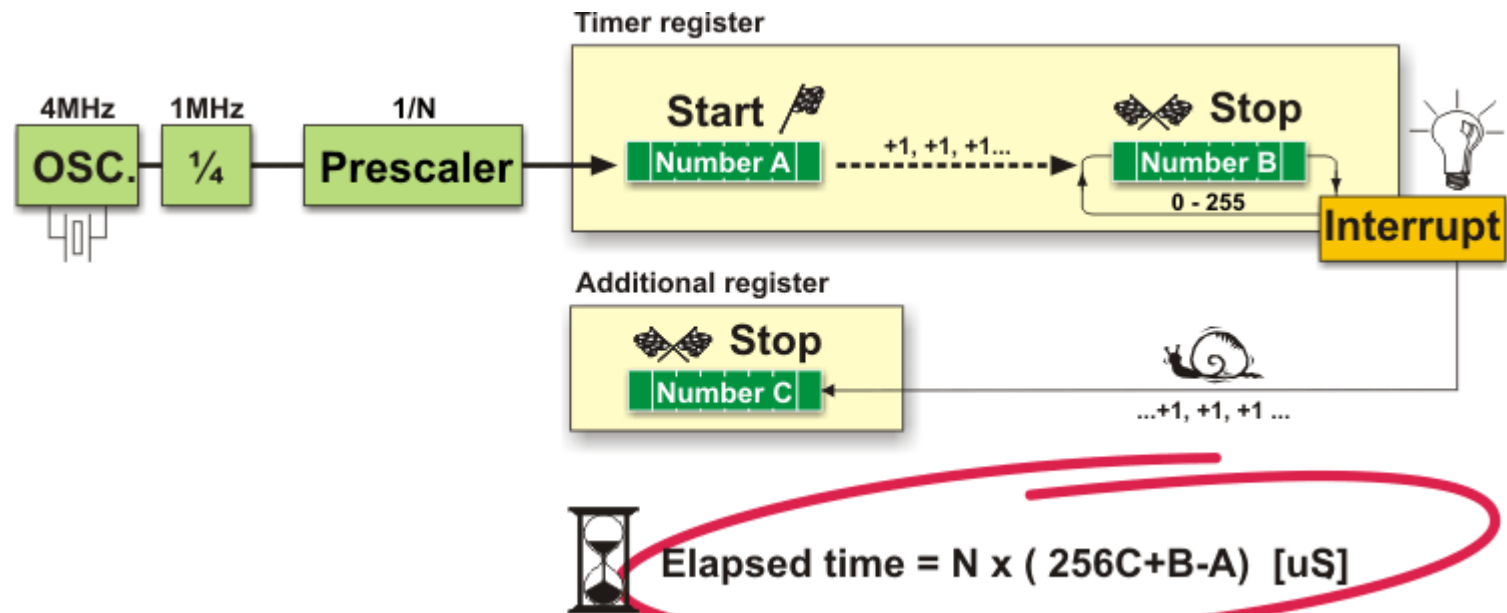
- A pre-scaler is a device used to reduce the frequency by a predetermined factor.
- An output pulse of a pre-scaler is generated after 1, 2, 4 or more pulses on its input.



Source: <https://learn.mikroe.com/ebooks/picmicrocontrollersprogramminginassembly/front-matter/introduction-to-the-world-of-microcontrollers/>

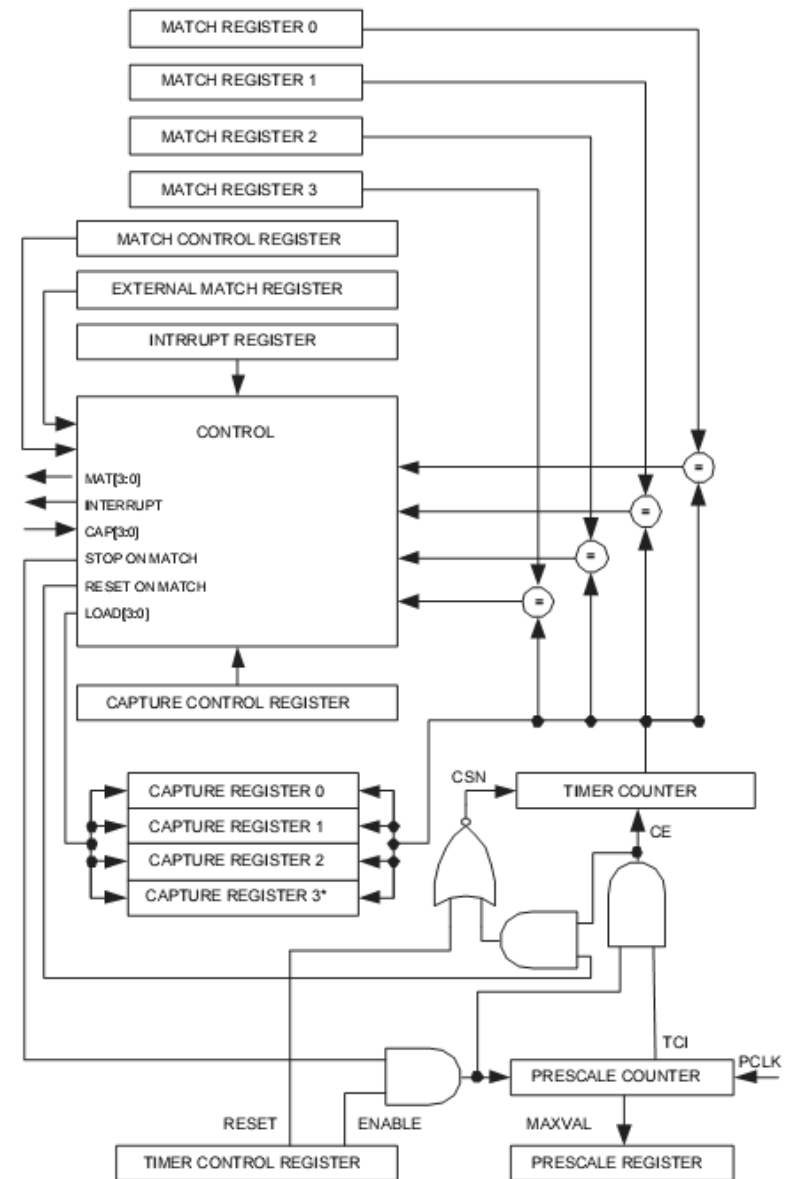
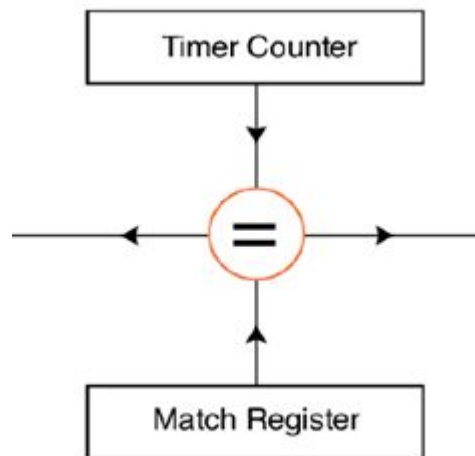
Using Interrupts to Measure Longer Periods

- If timer overflows, it can be programmed to generate an interrupt. This can be used to increment the value in another register.
- The timer can then automatically reset and counting will start from zero again.



Typical Features of a Timer

- Pre-scaler.
- Capture registers that can take snapshots.
- Match registers that allow interrupt, stop or reset when match happens.



* Note: that the capture register 3 cannot be used on TIMER0

Source: <http://www.scienceprog.com/timer-counters-in-lpc2000-microcontrollers/>